Laboratorio 1 · Los procesos desde el terminal

PSP · 2º DAM · Tema 1 · Viernes 30 de octubre de 2026

Cómo trabajamos

- Cada reto tiene una versión para **LINUX** y otra para **WINDOWS**: **todos tenéis que hacerlo en los dos sistemas**. Comparad los resultados y apuntad las respuestas.
- En Linux, abrid un **Terminal**. En Windows, abrid **PowerShell** (no hace falta que sea como administrador). Ningún reto necesita `sudo` ni permisos de administrador.
- Si un comando no funciona, leed el mensaje de error antes de preguntar: casi siempre os dice qué pasa.

Reto 1 · ¿Quién se está ejecutando?

LINUX

```
ps                  # procesos de este terminal
ps -ef              # todos los procesos
ps -e --no-headers | wc -l   # cuántos hay
top                 # en directo (q para salir)
```

WINDOWS

```
tasklist                # todos los procesos
(Get-Process).Count     # cuántos hay
Get-Process | Sort-Object CPU -Descending |
  Select-Object -First 5 Name, Id, CPU
# Ctrl+Mayús+Esc: Administrador de tareas
```

a) ¿Cuántos procesos hay en cada sistema? ¿Por qué creéis que hay tantos si solo tenéis abierto el terminal?

b) En Linux, ¿cómo se llama el proceso con PID 1? (`ps -p 1`). ¿Qué creéis que hace?

c) ¿Qué proceso usa más memoria en cada sistema? Linux: `ps aux --sort=-%mem | head -5` · Windows: ordenad por `WorkingSet64` en lugar de `CPU`.

Reto 2 · Un programa, varios procesos

LINUX

```
# el mismo programa, tres veces:
sleep 300 & sleep 300 & sleep 300 &
ps -o pid,comm              # ¿tienen el mismo PID?
kill <PID>                  # termino solo uno
ps -o pid,comm              # ¿siguen los otros?
pkill sleep                 # al acabar, los termino todos
```

WINDOWS

```
# el mismo programa, tres veces:
1..3 | ForEach-Object { Start-Process ping "-n 300 127.0.0.1" -WindowStyle Minimized }
Get-Process ping            # ¿tienen el mismo Id?
Stop-Process -Id <PID>      # termino solo uno
Get-Process ping            # ¿siguen los otros?
Stop-Process -Name ping     # al acabar, los termino todos
```

a) Procesos al lanzarlos: _____ · después de terminar uno: _____

b) Es el mismo programa: ¿por qué al terminar un proceso siguen los otros? Con esta idea, ¿por qué si una pestaña del navegador se cuelga no se cierran las demás?

Reto 3 · Padres e hijos

LINUX

```
echo $$                 # PID de mi terminal (bash)
ps -o pid,ppid,comm     # PID y PID del padre
pstree -p -s $$         # mis antepasados
bash                    # abro un bash dentro de bash
ps -o pid,ppid,comm     # ¿qué ha cambiado?
exit
```

WINDOWS

```
$PID                    # PID de esta PowerShell
(Get-CimInstance Win32_Process `
  -Filter "ProcessId=$PID").ParentProcessId
Get-Process -Id <PID_del_padre>
powershell              # PowerShell dentro de PowerShell
$PID
exit
```

a) ¿Quién es el padre de vuestro terminal en cada sistema?

---

b) Al abrir un terminal dentro de otro, ¿qué relación hay entre el PPID del nuevo y el PID del anterior?

---

c) En Linux, siguiendo la cadena hacia arriba, ¿a qué proceso se llega siempre?

Reto 4 · Crear, pausar y matar procesos

LINUX

```
sleep 300 &             # proceso en segundo plano
sleep 400 &
jobs                    # mis trabajos
ps -o pid,stat,comm     # fijaos en la columna STAT
fg %1                   # lo traigo al primer plano
#   Ctrl+Z  → lo detengo
ps -o pid,stat,comm     # ¿qué STAT tiene ahora?
bg %1                   # sigue en segundo plano
kill <PID>              # lo termino
kill -l                 # todas las señales
pkill sleep             # mata todos los sleep
```

WINDOWS

```
Start-Job { Start-Sleep 300 }   # trabajo en 2.º plano
Get-Job
Get-Process powershell          # ¿cuántos hay ahora?
Get-Job | Stop-Job
Get-Job | Remove-Job

$p = Start-Process ping -ArgumentList "-t","localhost" -PassThru
$p.Id                           # PID del nuevo proceso
Stop-Process -Id $p.Id          # lo termino
# Alternativa clásica:
taskkill /PID <PID> /F
```

a) En Linux, ¿qué letra aparece en `STAT` para un `sleep` que espera y para uno detenido con Ctrl+Z?

---

b) En Windows, al crear el trabajo con `Start-Job`, ¿aparece algún proceso nuevo? ¿Por qué?

---

c) ¿Qué diferencia hay entre `kill PID` y `kill -9 PID`? (Pista: `kill -l`, SIGTERM y SIGKILL).

Reto 5 · Los demonios

LINUX

```
systemctl list-units --type=service \
  --state=running --no-pager
ps -eo comm | grep 'd$' | sort -u
```

WINDOWS

```
Get-Service | Where-Object Status -eq Running
tasklist /svc /fi "imagename eq svchost.exe"
```

a) ¿Cuántos servicios (demonios) se están ejecutando en cada sistema? Escribid el nombre de tres y lo que creéis que hacen.

---

b) ¿Por qué muchos procesos de Linux terminan en «d»? ¿Qué tiene de especial `svchost.exe`?

Reto 6 · Mirar dentro de un proceso

LINUX

```
ls /proc | head
grep -E 'Name|State|PPid|Threads|VmRSS' \
  /proc/$$/status
```

WINDOWS

```
Get-Process -Id $PID | Select-Object Id,
  ProcessName, StartTime, WorkingSet64,
  @{n='Hilos'; e={$_.Threads.Count}}
```

a) ¿Qué datos guarda el sistema operativo de cada proceso? Relacionadlo con la diapositiva «¿Qué es un proceso?».

---

b) Mirad cuántos hilos tiene el navegador: `grep Threads /proc/<PID>/status` · `(Get-Process msedge)[0].Threads.Count`. ¿Un proceso puede tener varios hilos?

Reto final · ¿Cuántos procesos hacen falta para llenar la CPU?

Abrid el monitor (`top` en Linux pulsando **1** para ver cada núcleo, o el Administrador de tareas → Rendimiento → CPU con «procesadores lógicos»). Lanzad **un** proceso que solo gasta CPU y mirad qué pasa. Después lanzad tantos como núcleos tenga el equipo.

LINUX

```
nproc                    # núcleos
yes > /dev/null &        # uno
for i in $(seq $(nproc)); do
  yes > /dev/null &
done
pkill yes                # ¡no os olvidéis!
```

WINDOWS

```
$env:NUMBER_OF_PROCESSORS
Start-Job { while ($true) {} }       # uno
1..$env:NUMBER_OF_PROCESSORS | ForEach-Object {
  Start-Job { while ($true) {} } }
Get-Job | Stop-Job; Get-Job | Remove-Job
```

a) Con un solo proceso, ¿cuánta CPU total se usa? ¿Y con tantos como núcleos?

---

b) Si lanzáis el doble de procesos que núcleos, ¿se ejecutan todos a la vez de verdad? ¿Cómo lo consigue el sistema operativo?

Antes de iros: comprobad que no queda ningún `yes`, `sleep` o trabajo de PowerShell en marcha (`pgrep yes` · `Get-Job`).